

Reliable Structured-Decision LLMs for Applicant Ranking with a Non-Autoregressive System One Model

Jessie Angelica¹, Hasanul Fahmi Zuhri^{1,2,*}, Rusdianto Roestam¹

¹Faculty of Computer Science, President University, Bekasi, Indonesia

²Faculty of Artificial Intelligence & Frontier Technologies, UNITAR International University, Petaling Jaya, Malaysia

Abstract—LLM pipelines for applicant ranking typically combine free-text assessment with iterative listwise comparison, an approach that needs extra machinery against hallucinated claims and provides no built-in confidence signal. In September 2026, TypeSafe AI released Jev, the first “System One Model”: a non-autoregressive architecture that answers typed questions in one parallel pass and reports a calibrated confidence per answer, removing free-text generation and its hallucination risk by construction. No independent evaluation of this model class exists yet. The objective of this study is to close that gap by proposing and validating three architecture-level adaptations for the reliability problems an unmodified Jev introduces: criteria-grounded question design, multi-call ensemble averaging, and structured score decomposition as a diagnostic signal. The research method rebuilds an existing LLM-and-tournament applicant-ranking pipeline around Jev and evaluates it on 346 shortlisted (job, applicant) pairs across 10 job profiles. The results show that criteria-grounded design raises per-requirement answer confidence from 0.694 to 0.890 ($n = 869$, $p < 0.001$, rank-biserial $r = 0.844$) and pooled seniority/education confidence from 0.650 to 0.705 ($p < 0.001$); that three-call ensemble averaging, run independently and in parallel, recovers ranking convergence from 0.901 to 0.940 – close to the prior tournament-based system’s 0.957 – using half as many model calls, with internal coherence holding throughout ($\rho = 0.682$, $p < 0.001$); and that per-requirement score decomposition detects upstream shortlisting defects a single aggregate score cannot, flagging shortlisted applicants who score near-zero on a role’s core skill despite adequate scores elsewhere. Overall, a non-autoregressive structured-decision model can replace free-text assessment without losing reliability, provided its question design and aggregation match its architecture.

Keywords—Applicant Ranking, Confidence Calibration, Human Resources, Large Language Models, Structured Decision Models, System One Models.

I. INTRODUCTION

RECRUITMENT platforms increasingly use large language models (LLMs) to read resumes, summarize applicant fit, and compare applicants against a job profile, in place of manual screening or keyword search [1]–[4]. These systems are useful for the same reason they are risky: they generate free text that can assert a strength or weakness the source resume never supports. Prior work mitigates this with prompt-level constraints, retry-on-contradiction loops, post-hoc auditing [2], and iterative listwise tournaments with Plackett-Luce aggregation to turn per-applicant judgments

into a defensible ranking [1], [2]. Both exist because next-token sampling over an open vocabulary gives no structural guarantee against an ungrounded claim, and no native signal of the model’s confidence.

This paper asks whether answering predefined, typed questions instead of generating open text can replace free-text assessment without reintroducing the reliability problems it is meant to solve. Two gaps motivate this study. First, no independent, peer-reviewed evaluation of System One Models exists yet – Jev’s release [8] predates this manuscript by days – so every reliability claim currently comes from the vendor. Second, the recent literature on LLM calibration and self-consistency [5]–[7] targets autoregressive judges that produce a scalar score or a free-text verdict; none addresses a model whose primitive outputs are a bounded probability (Noul), a categorical distribution (Choice), or an ordinal distribution (Score) answered in one parallel pass, so its diagnostics and mitigations do not transfer directly.

This paper makes three architecture-level contributions, each validated empirically:

- 1) **Criteria-Grounded Question Design**: writing Score-type questions with concrete, situational evidence instead of bare ordinal labels. A paired ablation shows this raises confidence for the per-requirement and pooled seniority/education Score questions; the single, low- n overall-fit question does not reach significance alone. A separate internal-coherence check confirms per-requirement and overall-fit judgments agree.
- 2) **Multi-Call Ensemble Averaging**: combining several independent Jev calls per applicant – arithmetic mean for score-valued answers, confidence-weighted majority vote for categorical ones – to recover most of an iterative tournament’s ranking-order stability at roughly half the model calls, with no iterative ranking process needed.
- 3) **Structured Score Decomposition as a Diagnostic Signal**: per-requirement score granularity, unavailable to a monolithic free-text judgment, that detects and localizes a pre-existing upstream shortlisting defect a prose-only system could only find by manual inspection.

Section II reviews related work. Section III details the proposed pipeline’s architecture and the procedure behind each contribution. Section IV describes the experimental setup. Section V reports results, discusses them, and compares this

*Corresponding author: fahmi.zuhri@unitar.my.

system against the closest prior published system. Section VI concludes.

II. LITERATURE REVIEW

The system this paper extends combines LLM-driven free-text assessment with an active listwise tournament aggregated via Bayesian Plackett-Luce under Monte Carlo knowledge-gradient subset selection [1], [2]. Other systems apply LLMs to resume extraction and pointwise scoring inside retrieval-augmented or multi-agent architectures, without a comparative ranking step [3], [4]. None of this work departs from free text or free-form JSON as its output mode, so all of it inherits the hallucination-mitigation burden [2] addresses with a retry loop, and none reports a native confidence that could distinguish a well-supported judgment from a guess.

A separate literature asks whether an LLM’s stated confidence reflects its true correctness probability. Yaldiz et al. [5] find RL from verifiable rewards improves accuracy but makes decision-making LLMs overconfident. Wang et al. [6] find structured-output reliability varies sharply across models and prompts under a fixed schema. Halder and Hockenmaier [7] document substantial self-inconsistency in LLM-as-a-judge frameworks, mitigated by averaging judgments; a survey by Gu et al. catalogs reliability strategies for such judges, but every one surveyed still targets a free-text or scalar-score judge [11]. All of it treats the model as autoregressive, estimating confidence from token probabilities or resampling – an assumption Jev’s natively-trained, non-autoregressive confidence field does not share, so these findings cannot be assumed to transfer. This paper verifies that independently.

Jev is explicitly not a conventional language model: TypeSafe’s own documentation states plainly that “Jev is not a language model. You give it a text and typed questions, and it answers each one with a confidence. It does not write text” [17]. It takes a text state and typed questions (Noul, Choice, or Score) and returns, in one parallel pass, a calibrated probability distribution over each answer – no tokens sampled, output priced at zero since none is generated [8]. TypeSafe calls this a “System One Model,” trained with a proprietary method it names Reinforcement Learning for Calibrated Decisions (RLCD) on a purpose-built parallel sampler [8]; the underlying architecture (encoder-only, decoder-only, or otherwise) is undisclosed in every source consulted. Mapping a fixed input to a distribution over a closed answer set in one forward pass, rather than generating a variable-length sequence, is consistent with a classifier-style, encoder-like design – an inference from behavior, not a confirmed fact. Jev was created by TypeSafe co-founder/CEO Diogo Almeida, who “co-invented RLHF and InstructGPT, the methods that lead to ChatGPT and GPT4” [18]; TypeSafe launched it publicly on September 15, 2026 with a \$40M seed round [8]. As used here (`jev-latest`), it accepts up to 64,000 tokens per request (32,000 for the state plus the longest question) [19], responds in 70–500ms, and costs \$0.042 per million input tokens with output free [8].

No paper reviewed here evaluates a System One Model; to the authors’ knowledge this is the first independent study of this model class, whose own claims live only in vendor-controlled or community sources at the time of writing –

the announcement [8], an integration guide [14], a community write-up [15], and a release summary [16] – none peer-reviewed. This work sits in applicant ranking, already served by a directly comparable free-text predecessor [1], [2], making an architecture-level comparison possible, not just a benchmark-level one.

III. METHODOLOGY

A. Pipeline Overview

Fig. 1 shows the end-to-end pipeline, with C1–C3 marking where each contribution intervenes. Its first three stages are unchanged from the prior system [2], since none generate free text: Job profiles are loaded as plain text and resumes parsed from PDF, an LLM (Qwen2.5-14B-Instruct) extracts a normalized skill list including any certification, seniority, and education requirement, and an applicant is shortlisted only if enough technical skills match by cosine similarity above a fixed threshold.

The pipeline changes at the next stage, where the prior system’s free text lived: instead of a free-text assessment ranked by an iterative tournament, it sends n independent calls to Jev per shortlisted pair. Each call gives Jev one state (job profile title and description, extracted skills, resume text) and typed questions answered in one pass: an overall-fit Score, a hiring-recommendation Choice, a minimum-qualifications Noul, one Score per technical requirement, one Score per stated seniority or education requirement, and one Noul per stated certification. Every Score question offers a fixed set of ordinal levels (Section III-B), linearly mapped to 0–100 for reporting – this range is the mapping’s resolution, not a continuous per-call judgment. The n calls are combined (Section III-C), applicants ranked by the combined score, and a diagnostic check (Section III-D) reads per-requirement scores afterward for upstream errors. Skill/job profile extraction runs on a conventional autoregressive LLM; Jev does not – it samples no tokens, only typed answers in one forward pass.

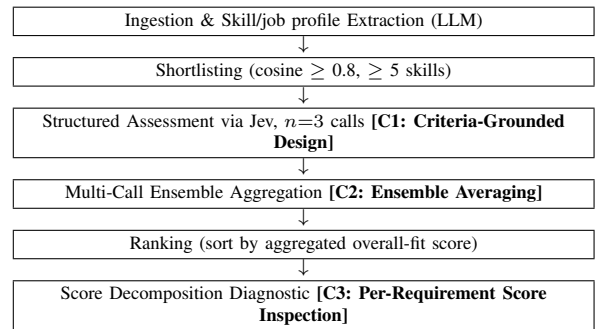


Fig. 1. End-to-end pipeline architecture, annotated with the three contributions (C1–C3) at their point of intervention.

B. Criteria-Grounded Question Design

An initial implementation defined every Score question’s five levels with bare numeric labels (“0,” “25,” “50,” “75,” “100”), with no description of what each level meant – asked to pick a level with no stated meaning, Jev had to guess the boundary, and guessing lowers confidence. The fix

describes each level as a concrete, situational fact instead: e.g., distinguishing a requirement “mentioned only as a bare skill-list item” from one “used substantively in a real role or project with clear responsibility,” so the model matches evidence to a description rather than guessing.

To check that this fix, not sampling noise, caused the confidence gain, a paired ablation compared the two designs on identical input: for a sampled subset of already-assessed pairs, a second call was issued against the same state under the original bare-label design, and confidence compared pairwise. The concrete side is the confidence already recorded in production (a mean across 3 calls); the bare-label side is a single fresh call – this shrinks the concrete side’s variance but does not bias either mean, so it cannot manufacture the gain reported. The primary comparison is the per-technical-requirement questions, with enough paired observations to detect the effect reliably; a Wilcoxon signed-rank test with rank-biserial effect size was applied.

The same fix generalizes to the seniority and education Score questions, which had the same bare-label problem. Rewriting the levels to name concrete evidence – years/level, domain, or role match for seniority; field and degree-level match for education – follows the same logic, tested with a second paired ablation on all pairs whose job profile states either requirement. One alternative was rejected: generating levels per job profile with an LLM, since that reintroduces free-text generation and makes scores incomparable across differently worded job profiles.

C. Multi-Call Ensemble Averaging

A single Jev call is a single sample, carrying noise that can shift a ranking. Since calls are independent and cheap, the pipeline issues n calls per pair – three by default, matching [2]’s stability-repeat count – combined per field: numeric fields by arithmetic mean, the recommendation by majority vote (ties toward higher mean confidence), and binary answers by simple majority, defaulting to the conservative (negative) outcome on a tie.

Averaging could also hide real disagreement rather than remove noise: ranking convergence was measured both from single, unaggregated calls (Kendall- τ across three per-pair rankings) and from three three-call averages. If averaging only hid disagreement, the second would not consistently beat the first.

D. Structured Score Decomposition as a Diagnostic Signal

Because Jev reports a separate score per extracted requirement rather than a single free-text verdict, a job profile’s shortlisted pool can be inspected at the level of individual skills, not only an aggregate score. This granularity is used diagnostically: a requirement is informative only if the pool’s median score on it is reasonably high; an applicant scoring near-zero on most informative requirements, despite being shortlisted, is flagged – not because Jev judged them harshly, but because peers could clear a bar this applicant could not.

IV. EXPERIMENTAL SETUP

The pipeline is evaluated on the prior system’s corpus and shortlist configuration [2]: 501 resumes across 10 synthetically authored job profiles, shortlisted at a cosine-similarity threshold of 0.8 and a minimum of five matched technical skills, yielding 346 shortlisted (job, applicant) pairs. Table I lists the full pipeline configuration; these settings were held fixed across every evaluation, isolating the assessment-stage change as the one independent variable between the two systems. Four evaluation procedures are applied: test-retest reliability (three independent single calls per pair); internal coherence (per-requirement scores against overall fit via Spearman correlation); ranking convergence (Kendall- τ , Section III-C, from single calls and three-call averages); and the criteria-design ablations of Section III-B. All tests are two-sided except where a directional hypothesis is stated explicitly, and every effect size accompanies, rather than substitutes for, its significance test.

TABLE I
PIPELINE CONFIGURATION.

Parameter	Value
Assessment model	Jev (jev-latest), TypeSafe AI
Skill/job profile extraction model	Qwen2.5-14B-Instruct (4-bit, Ollama)
Skill embedding model	sentence-transformers/all-MiniLM-L6-v2
Corpus	501 resumes, 10 job profiles
Shortlisted (job, applicant) pairs	346 (338–346 run-to-run)
Skill-match threshold / min. matches	cosine ≥ 0.8 / ≥ 5 job skills
Independent calls averaged per assessment (n)	3
Score question levels	5 (concrete situational criteria)
Test-retest repeats per pair	3
Criteria-design ablation sample	40 pairs (Score); 169 pairs (seniority/education Score)

V. RESULTS AND DISCUSSION

A. Test-Retest Reliability and Internal Coherence

Across 346 pairs $\times$ 3 repeats, the overall-fit score’s within-pair standard deviation (0–100 scale) has mean 0.82, and the recommendation is identical across all 3 repeats for 95.1% of pairs – both indicate low single-call noise before any aggregation. Per-requirement scores correlate with the overall fit score ($\rho = 0.682$, $p < 0.001$, $n = 346$).

$\rho = 0.682$ falls short of 1.0 not from a reliability defect but from how the numbers are produced: Jev answers overall-fit and every per-requirement question as independent items in the same call, never computing one from the other, so they are two separate readings of the same evidence. Overall fit also weighs things an unweighted skill average cannot, such as seniority fit, and single-call noise pulls the correlation down further. A correlation near zero would signal incoherent judgments; $\rho = 0.682$ is what two related but distinct judgments should look like.

B. Criteria-Grounded Question Design

Table II pairs each question asked twice under the two criteria designs and reports mean confidence under each plus

TABLE II
CRITERIA-DESIGN ABLATIONS: CONCRETE COMPARED WITH BARE-LABEL CRITERIA, PAIRED BY (JOB, APPLICANT) PAIR.

Question group	n	Conf. (concrete/other)	p (r)
Per-requirement	869	0.890 / 0.694	< 0.001 (0.84)
Seniority	66	0.583 / 0.585	0.597 (0.08)
Education	146	0.760 / 0.679	< 0.001 (0.45)
Pooled (sen. + edu.)	212	0.705 / 0.650	< 0.001 (0.34)

the Wilcoxon result. The per-requirement questions show a large, highly significant gain ($r = 0.844$, $p < 0.001$, $n = 869$). Seniority and education reach significance pooled and for education alone but not seniority alone ($p = 0.597$, $n = 66$); the direction matches, but the pooled effect is smaller ($r = 0.34$ vs. $r = 0.844$), since per-requirement pools many skill-match judgments per pair while seniority and education contribute only one each, so seniority’s null result pulls the pooled effect down. That null result is not noise: education’s criteria check explicit facts a resume states or does not (a degree, a field), which concrete wording sharpens directly, while seniority requires inferring years of experience and domain relevance at once – a harder judgment bare labels do not meaningfully worsen.

C. Multi-Call Ensemble Averaging

τ_1 is from single unaggregated calls, τ_3 from three-call averages (Section III-C). As shown in Table III’s $\tau_1 \rightarrow \tau_3$ columns, three-call averaging raises mean convergence from 0.901 to 0.940, with 9 of 10 profiles improving (one unchanged) – improvement across nearly every profile confirms the aggregation recovers genuine stability, not just smoothing over disagreement.

D. Structured Score Decomposition Diagnostic

TABLE III
RANKING CONVERGENCE (KENDALL- τ) BY JOB PROFILE, SINGLE-CALL AND 3-CALL-AVERAGED.

Job profile	n	τ_1	τ_3
it-security-engineer	16	0.778	0.900
data-scientist-eng.	27	0.848	0.890
backend-engineer	22	0.896	0.954
frontend-engineer	13	0.880	0.897
devops-engineer	58	0.923	0.956
full-stack-engineer	33	0.914	0.944
data-engineer	76	0.937	0.958
ui-ux-designer	23	0.947	0.984
java-developer	21	0.949	0.949
cloud-engineer	57	0.933	0.963
Mean (all 10)	–	0.901	0.940

Table III gives each profile’s ranking convergence from single and three-call-averaged calls. The diagnostic value of per-requirement decomposition is demonstrated concretely: the informative-requirement check flagged 8 of 76 data-engineer applicants (11%) near-zero specifically on Python, and 2 of 33 full-stack-engineer applicants (6%) near-zero on REST APIs and Node.js despite adequate scores elsewhere – evidence the skill-matching stage, not Jev, admitted these applicants.

E. Efficiency

Across all 1,038 calls in the test-retest data ($346 \text{ pairs} \times 3 \text{ repeats}$), a single Jev call takes a mean of 1.20s (range 0.73–

3.33s), for a total of 20.7 minutes of compute time across the full corpus – call latency only, not skill/job profile extraction.

F. Comparison with the Prior Tournament-Based System

TABLE IV
COMPARISON AGAINST THE CLOSEST PRIOR PUBLISHED SYSTEM [2].

Dimension	Prior system [2]	This work
Assessment	Free-text strengths/weaknesses	Typed Score/Choice/No/Unanswered in one parallel pass
Output shape	+ retry-on-contradiction Free prose, validated against a schema after generation	Predefined typed answers; schema violation impossible by construction
Hallucination handling	Live contradiction-check-and-regenerate loop	Not applicable: no free text to hallucinate
Identifier robustness	Positional tokens + schema-constrained ranking + retry	Not needed: one call per applicant, no identifier passing
Ranking	Iterative MC-KG tournament + Plackett-Luce	Direct sort on aggregated score
Convergence	0.957	0.940 (3-call average)
Total calls (corpus)	$\approx 2,061$ (348 assess. + 1,713 tournament)	1,038 (346×3), 0 tournament
Generation model	Qwen2.5-14B-Instruct, 4-bit, local via Ollama	jev-latest, hosted by Type-Safe AI
Confidence signal	Faithfulness 0.880 (entailment)	Calibrated confidence 0.890 (requirement questions); different construct
Explanation	Strengths/weaknesses narrative	Score + confidence, no narrative

As shown in Table V, this system is not better in every way: its ranking is slightly less stable, and Faithfulness and per-answer confidence measure different things (groundedness vs. decision stability), so neither proves it is “more accurate.” It wins on cost – half the calls, no tournament infrastructure, schema violation impossible by construction rather than merely reduced – and offers a native, calibrated confidence the prior system had no way to provide. It also parallelizes: the prior system’s 1,713 tournament calls form a sequential chain, each iteration waiting on the last, while this system’s calls are independent.

VI. CONCLUSION

This paper replaced the free-text assessment and iterative tournament-ranking stages of an existing pipeline with Jev, TypeSafe AI’s first System One Model, and validated three adaptations – criteria-grounded question design, multi-call ensemble averaging, and structured score decomposition as a diagnostic signal – each motivated by a reliability problem observed in practice. The result is not a uniform improvement (Table V): it trades a little ranking stability and all free-text interpretability for roughly half the model calls, no tournament infrastructure, and a native calibrated confidence the prior system never had. Criteria-grounded design is strongly validated for the per-requirement Score questions ($r = 0.844$, $p < 0.001$, $n = 869$) and, with a smaller pooled effect, for seniority and education ($r = 0.34$); the single, low- n overall-fit question does not reach significance alone. Score decomposition’s diagnostic value is confirmed quantitatively, surfacing a shortlisting-stage defect the prior system could only have found by manual reading.

A limitation is that this work trades away the prior system’s free-text interpretability – a score and a confidence, not a narrative, and per-requirement scores only partly substitute.

A second is that the shortlisting stage, inherited unchanged from the prior system, counts skill matches toward a minimum without distinguishing a role’s core stack from peripheral skills, so an applicant can be admitted on utility skills alone (e.g., Docker, Kubernetes, AWS) while scoring near-zero on the role’s core requirements – the full-stack-engineer case in Section V-D. Score decomposition detects this defect but cannot correct it, since fixing it would require modifying the shortlisting stage both pipelines share.

Future work includes a lightweight explanation layer – surfacing the lowest-scoring items as a structured rationale – to partially restore interpretability without reintroducing free text; and separating a role’s core technical skills from peripheral ones during job profile extraction, so shortlisting stops admitting applicants on utility skills alone.

ACKNOWLEDGMENT

The authors acknowledge the use of NVIDIA GeForce RTX 4070 Ti SUPER GPU hardware for the local skill/job profile-extraction stage, and the open-source LangGraph, sentence-transformers, and PyTorch libraries.

REFERENCES

- [1] K. A. Yuksel, A. B. Anees, A. Elneima, S. Hewavitharana, M. Al-Badrashiny, and H. Sawaf, “Agentic AI for Human Resources: LLM-Driven Candidate Assessment,” in *Proc. 19th Conf. Eur. Chapter Assoc. Comput. Linguistics (EACL)*, vol. 3: System Demonstrations, Rabat, Morocco, Mar. 2026, pp. 341–348.
- [2] J. Angelica and H. F. Zuhri, “Hallucination-Aware Self-Correction for LLM-Based Applicant Assessment,” unpublished manuscript, 2026.
- [3] A. C. Tran and N. C. Tran, “A recruitment support system based on Large Language Models and Retrieval-Augmented Generation,” *Procedia Computer Science*, vol. 269, pp. 259–268, 2025.
- [4] F. P.-W. Lo, J. Qiu, Z. Wang, H. Yu, Y. Chen, G. Zhang, and B. Lo, “AI Hiring with LLMs: A Context-Aware and Explainable Multi-Agent Framework for Resume Screening,” arXiv:2504.02870, 2025.
- [5] D. N. Yaldiz, E. Spiliopoulou, Z. Qi, S. Varia, S. Doss, and N. Pappas, “Balancing Classification and Calibration Performance in Decision-Making LLMs via Calibration Aware Reinforcement Learning,” arXiv:2601.13284, 2026.
- [6] G. Wang, J. Yu, X. Zhang, D. Jiang, Y. Song, T. Deb, X. Liu, and P. He, “STED and Consistency Scoring: A Framework for Evaluating LLM Structured Output Reliability,” arXiv:2512.23712, 2025.
- [7] R. Haldar and J. Hockenmaier, “Rating Roulette: Self-Inconsistency in LLM-As-A-Judge Frameworks,” in *Findings of the Assoc. Comput. Linguistics: EMNLP 2025*, 2025.
- [8] D. Almeida, “Introducing System One Models & Jev,” Type-Safe AI Company Blog, Sep. 15, 2026. [Online]. Available: <https://typesafe.ai/blog/introducing-system-one-models-and-jev>
- [9] J. Gu *et al.*, “A survey on LLM-as-a-judge,” *The Innovation*, vol. 7, no. 6, art. 101253, 2026.
- [10] S. Runkle and H. Lovell, “Building a Harness with Jev,” LangChain Blog, Sep. 17, 2026. [Online]. Available: <https://www.langchain.com/blog/building-a-harness-with-jev>
- [11] P. Burnhill, “TypeSafe Jev – Project Knowledge Resource,” GitHub Gist, Sep. 16, 2026. [Online]. Available: <https://gist.github.com/pjburnhill/adf8d28efcad9df037bfdece178ef965>
- [12] “Jev – TypeSafe’s model returns typed decisions, not text,” AI/TLDR, Sep. 15, 2026. [Online]. Available: <https://ai-tldr.dev/releases/typesafe-jev/>
- [13] “TypeSafe (Jev),” Pydantic AI Documentation. [Online]. Available: <https://pydantic.dev/docs/ai/models/typesafe/>
- [14] “Team,” TypeSafe AI, accessed Sep. 2026. [Online]. Available: <https://typesafe.ai/team>
- [15] “Models,” TypeSafe AI Documentation, accessed Sep. 2026. [Online]. Available: <https://docs.typesafe.ai/models>